

HERALD: High-Throughput Block Diffusion LLM Serving via CPU-GPU Cooperative KV Cache Retrieval

Omin Kwon
Seoul National University
Seoul, Republic of Korea

Doyeon Kim
Seoul National University
Seoul, Republic of Korea

Jongseok Park
UC Berkeley
Berkeley, CA, USA

Seung Yul Lee
Seoul National University
Seoul, Republic of Korea

Ion Stoica
UC Berkeley
Berkeley, CA, USA

Jae W. Lee
Seoul National University
Seoul, Republic of Korea

Abstract—The KV cache dominates GPU memory in long-context LLM serving: it grows with context length, crowds out batch capacity, and leaves GPU compute idle. Offloading the cache to CPU DRAM restores capacity, but the limited PCIe bandwidth forces state-of-the-art offloading systems to pair it with sparse attention, fetching only a small critical subset of the cache to the GPU. These systems, however, follow the KV access pattern of autoregressive decoding, in which the critical set changes at every token: selection and fetching recur at every decoding step, and throughput remains capped by PCIe bandwidth rather than by either processor. Block diffusion LLMs (block dLLMs), which decode a block of B tokens over T denoising steps, exhibit a different KV access pattern that opens a new opportunity for offloading. Recent sparse block dLLM methods have shown that sparse inference separates into a selection phase that scans the full KV cache once per block and a denoising phase that reuses the selected small subset T times. This asymmetry aligns with the compute and memory asymmetry of a CPU-GPU system, making it advantageous to run selection on the CPU and denoising on the GPU: the critical KV cache then crosses PCIe only once per block, removing the interconnect as the bottleneck. We present HERALD, to our knowledge the first KV offloading system designed for block dLLMs. HERALD resolves the two obstacles of this mapping, the serialized dependency between the phases and the compute-bound B -query selection on the CPU, by overlapping the phases with a draft prefix built from step-0 logits, reducing the selection cost with a single representative query, and executing both as a dual-stream pipeline over double-buffered sparse KV pools. On two production block dLLMs, HERALD sustains near-lossless accuracy at a 5% KV budget and reaches up to $2.28\times$ the decode throughput of GPU-only serving, with gains that widen with context length.

I. INTRODUCTION

Block diffusion large language models (block dLLMs) are emerging as a model architecture that provides higher decode throughput than autoregressive large language models (AR LLMs). A block dLLM decodes a block of B tokens in parallel over T denoising steps, and generation remains autoregressive at the block level. Recent block dLLMs reach accuracy close to AR models at the same scale [3], [4], and production systems such as Gemini Diffusion [9] have adopted the block dLLM architecture. At long contexts, however, block dLLMs suffer the same throughput loss as AR LLMs. The KV cache grows in proportion to the context length and occupies most of the

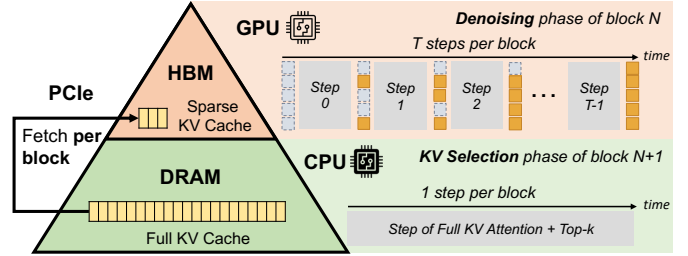


Fig. 1: HERALD’s mapping of block dLLM sparse-attention inference onto a heterogeneous CPU-GPU system: the KV selection phase runs once per block on the CPU over the full KV cache, the T -step denoising phase runs on the GPU over the critical KV cache, and the critical KV cache crosses PCIe once per block.

HBM, which caps the batch size well before GPU compute saturates.

Existing KV offloading systems for AR LLMs, which address this capacity limit, raise throughput, but the gain is still capped by per-step KV fetching over PCIe. Prior work offloads the KV cache to CPU DRAM [28]; fetching the full KV cache back at every step is PCIe-bound, so state-of-the-art systems select only the critical KV cache and run sparse attention on the GPU [18], [31], [37]. Even for the critical KV cache, however, the per-step fetching throughput remains PCIe-bound below the GPU model forward throughput, capping the decode throughput.

For this KV fetching bottleneck, block dLLMs provide a structural opportunity that AR LLMs do not have. MAGE [16], a sparse-attention algorithm designed for block dLLMs, selects the critical KV cache once with exact attention at the first step of a block, and then reuses the critical KV cache for sparse attention over the remaining $T-1$ steps. This reuse is valid because the critical KV cache exhibits strong temporal consistency across denoising steps [16], [29], [33]. Through this reuse, block dLLM inference separates into a *KV selection phase*, which requires the full KV cache and runs once per block, and a *denoising phase*, which requires only the critical KV cache and runs $T-1$ times.

HERALD proposes to map the selection phase to the CPU and the denoising phase to the GPU (Figure 1). Under this mapping, the critical KV cache crosses PCIe only once per block: the fetching volume falls to $1/T$ of that of per-step fetch systems, which lifts the KV fetching ceiling above the GPU model forward throughput and removes KV fetching as the bottleneck. The two phases also align with the asymmetric resources of a heterogeneous CPU–GPU system, which consists of the CPU, the GPU, and the PCIe link between them. In memory capacity, the selection phase, which requires the full KV cache, corresponds to the large CPU DRAM, while the denoising phase, which requires only the far smaller critical KV cache, corresponds to the small GPU HBM. In compute capability, if the two phases overlap, the CPU has to finish only one selection while the GPU performs T denoising steps, so the $1:T$ frequency ratio compensates for the CPU’s lower compute and memory bandwidth resources.

Realizing this mapping, however, requires co-designing the new sparse selection algorithm with the offloading system, because of two obstacles. First, the denoising phase in the MAGE algorithm can start only after the selection phase completes. This sequential dependency serializes the two phases even when they are placed on the CPU and the GPU, which holds throughput at about half of what full overlap allows. Second, the selection phase is exact attention with B queries in GEMM form, which makes it compute-bound on the CPU. Attention in an AR LLM uses a single query and is memory-bound, whereas a block dLLM must process B queries. The resulting low CPU attention throughput makes the selection phase the bottleneck even within a window of T steps.

This paper presents HERALD, which resolves both obstacles using structural properties of block dLLMs. The first modification, draft-based lookahead, removes the sequential dependency. Taking the argmax at every position of block n ’s step-0 logits yields a draft of block n without an additional forward pass, and HERALD uses this draft as a temporary prefix so that block $n+1$ ’s selection overlaps with block n ’s denoising. The second modification, representative-query selection, reduces the CPU computation. At the time of selection, all B positions of the next block carry the same [MASK] embedding and differ only in their RoPE offsets, so HERALD replaces the B queries with a single representative query at the center of the block and reduces the selection computation in proportion to the query count. The selection quality lost by the two modifications is recovered by enlarging the sparsity budget about $1.7\times$, and the only cost is the batch size shrinking by the same factor, which is negligible next to the throughput gain that the two modifications provide.

HERALD executes the two phases as a dual-stream pipeline over double-buffered sparse KV pools. While the main stream denoises block n with the critical KV cache in the active pool, the retrieval stream runs block $n+1$ ’s selection phase as a layer-wise CPU–GPU cooperative pipeline. In each layer, the CPU scans the full KV cache with a kernel that fuses attention and top- k selection while the GPU executes that layer’s FFN, and the gather and DMA transfer of the selected KV entries

are hidden behind the FFN and the next layer’s computation. When block n ’s denoising ends, the idle pool already holds block $n+1$ ’s critical KV cache, so the next block starts as soon as the two pool pointers are swapped.

We implement HERALD on SGLang [40] and evaluate it on two production block dLLMs, SDAR-8B-Chat and LLaDA 2.0-mini 16B. On LongBench [2], HERALD reaches near-lossless accuracy at a 5% KV budget. Peak decode throughput is $1.81\text{--}1.82\times$ that of the Dense GPU-only baseline at 16K and $1.89\text{--}2.28\times$ at 32K, so the gap widens as the context grows. InfiniGen [18], which selects the critical KV cache with a proxy instead of exact attention, and MAGE-Offload, which applies the MAGE selection algorithm directly to the offloading setting, both remain below Dense.

We make the following contributions:

- We identify that the KV access pattern of block dLLMs, unlike that of AR LLMs, separates sparse-attention inference into a KV selection phase and a denoising phase, and show that mapping the two phases onto a heterogeneous CPU–GPU system eliminates the per-step KV fetching bottleneck of AR-style offloading.
- We identify the two obstacles of this mapping, the sequential dependency between the phases and the compute-bound dLLM attention on the CPU, and resolve them with two algorithm modifications: draft-based lookahead, which overlaps the phases with a draft prefix built from step-0 logits, and representative-query selection, which cuts the CPU selection computation by $B\times$.
- We present HERALD, to our knowledge the first KV offloading system designed for block dLLMs, which realizes this algorithm–system co-design as a dual-stream pipeline over double-buffered sparse KV pools with a fused CPU selection kernel.
- On two production block dLLMs, HERALD sustains near-lossless accuracy at a 5% KV budget and reaches up to $2.28\times$ the decode throughput of GPU-only serving.

II. BACKGROUND

A. Block Diffusion LLM Inference

Autoregressive (AR) LLMs generate one token per sequence with each model forward pass. Each forward pass accesses the model weights and the growing prefix KV cache to produce only one new token per sequence, resulting in low arithmetic intensity and underutilized GPU compute [14], [19], [30].

Diffusion LLMs (dLLMs) relax this token-level dependency by jointly refining multiple token positions over a sequence of denoising steps [7], [8], [22], [24], [27], [38]. Early dLLMs, however, apply bidirectional attention over the entire sequence at every step, preventing the KV states of generated tokens from being reused as a fixed cache and requiring repeated processing of the full sequence.

Block diffusion restores compatibility with KV caching by partitioning the output into fixed-size blocks of B tokens and generating the blocks autoregressively while applying diffusion-based parallel decoding within each block [1], [3], [4], [15], [35].

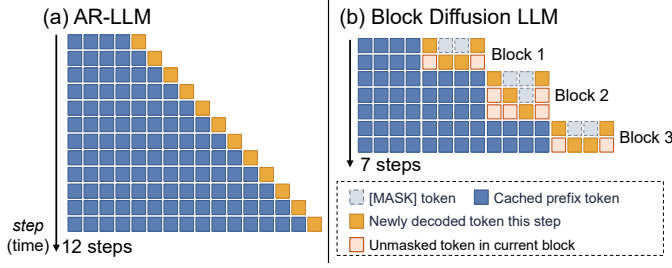


Fig. 2: Comparison of decoding processes. (a) AR LLM generates one new token per step. (b) Block diffusion LLM refines a block over multiple denoising steps, progressively accepting predicted tokens within the current block. Tokens in the current block become reusable cached prefix tokens only after the block is completed.

As illustrated in Figure 2, each block begins with B [MASK] tokens and is refined over T denoising steps. At each step, tokens attend bidirectionally within the current block and causally to the cached KV states of previously completed blocks. The scheduler accepts predicted tokens for a subset of masked positions, while the remaining positions stay masked for subsequent refinement [13], [36]. After T steps, all positions contain accepted tokens and the block is completed; its KV states are then computed and appended to the prefix KV cache. Generation then proceeds to the next block. Consequently, the KV cache of completed prefix blocks remains unchanged throughout the T denoising steps of a block, although the queries within the current block evolve as more predicted tokens are accepted. We call a position whose predicted token has been accepted an *accepted token position*, and a block in which every position contains an accepted token a *completed block*.

B. KV Cache Offloading in AR LLMs

In AR LLMs, long-context serving throughput is limited by GPU memory capacity. Each generated token appends new KV states to the prefix cache, so the KV cache grows linearly with context length and comes to occupy most of GPU HBM at long contexts. Decode throughput is obtained by enlarging the batch size until GPU compute is saturated; once the KV cache dominates HBM, the batch size can no longer scale to that point, and throughput is dictated by memory capacity rather than compute.

To remove this capacity constraint, many systems offload the KV cache to CPU DRAM, whose capacity is substantially larger than GPU HBM [11], [18], [28]. The model weights remain on the GPU while only the KV cache resides in CPU memory, so the batch size can scale up to the CPU memory limit rather than the HBM limit. With the resulting large batches, the GPU’s FFN computation can saturate GPU compute.

Under KV offloading, the throughput bottleneck is the attention computation, which requires the full prefix KV cache (*exact attention*). One strategy performs exact attention

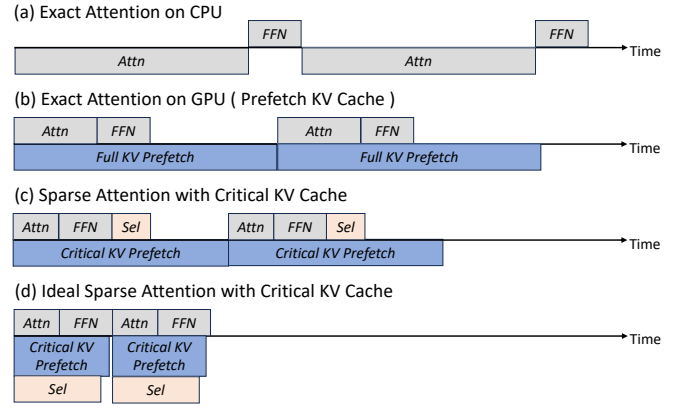


Fig. 3: Decoding methods under KV cache offloading in AR LLM.

directly on the CPU, where the full cache resides (Figure 3(a)); however, CPU memory bandwidth is roughly an order of magnitude lower than GPU HBM [10], [25], so the attention time inflates and throughput degrades [11], [12], [28]. To avoid this CPU attention bottleneck, the KV cache can instead be prefetched layer by layer so that the GPU performs the attention (Figure 3(b)) [28], but the PCIe interconnect between CPU and GPU then becomes the bottleneck: the transfer time far exceeds the model forward time, again degrading throughput.

To avoid this attention-induced throughput degradation under KV offloading, many works adopt sparse attention [18], [31], [37]. Sparse attention attends only to the small critical subset of the KV cache that matters for generating each token, instead of the full cache. This substantially reduces the volume of KV cache transferred from CPU to GPU (Figure 3(c)), thereby increasing decode throughput.

Sparse attention requires a *selection* process that identifies the critical KV cache and a *fetching* process that brings the selected KV cache to the GPU. InfiniGen [18] performs selection by predicting the critical KV entries of layer i from the attention input of layer $i - 1$, and prefetches them while the preceding layer executes, overlapping communication with computation. ShadowKV [31] keeps a compact low-rank representation of keys and outlier entries on the GPU for lightweight selection, and prefetches only the corresponding value entries on demand from CPU memory. SpeContext [37] decouples selection from the target model’s forward pass using a distilled retrieval model, and prefetches only the difference between consecutive selections to reduce the transfer volume.

Consequently, the decode throughput gain of a sparse-attention-based offloading system grows as less of the selection and fetching overhead is left exposed on the critical path unhidden behind the model forward pass. Any exposed overhead adds directly to the per-step latency (Figure 3(c)), eroding the throughput gain that the enlarged batch size provides; in the ideal case, both overheads are fully hidden and the latency is bounded by the model forward alone (Figure 3(d)). The

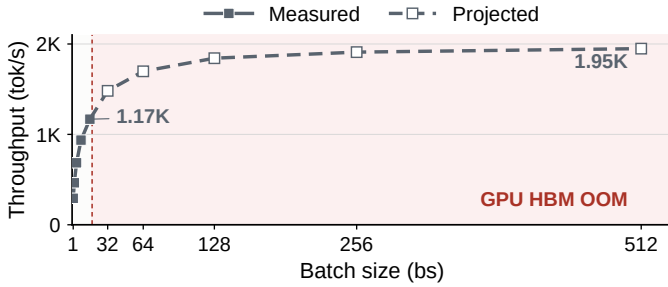


Fig. 4: Throughput analysis for LLaDA 2.0-mini at 32K context. HBM-limited decode throughput scaling of full-KV GPU-only execution.

throughput of existing systems is therefore determined by how much they reduce the two overheads and how well they hide the rest behind the model forward pass.

III. MOTIVATION

This section shows that block dLLMs, like AR LLMs, require KV offloading for high-throughput inference, and describes the opportunity that a new structural property of block dLLMs provides to achieve higher throughput than existing AR-style KV offloading systems. All measurements use LLaDA 2.0-mini [3] with a 32K-token context, block size $B = 32$, and $T = 20$ denoising steps on the same CPU-GPU platform as Section VI-A, and all throughput numbers refer to the decode phase.

A. KV Offloading Is Also Required in Block dLLMs

Block dLLMs are equally constrained by GPU memory capacity at long contexts, and therefore equally require KV offloading for high throughput inference. The only difference from AR LLMs is that KV caches are appended to the prefix cache block by block rather than token by token; the total KV cache held by a request is identical to that of an AR LLM at the same context length. Consequently, the same capacity problem applies: the KV cache occupies HBM and limits batch-size scaling. Figure 4 shows this on LLaDA 2.0-mini at 32K: full-KV GPU-only execution runs out of HBM at 1.17k tokens/s, short of the 1.95k tokens/s upper bound it could reach with unlimited HBM while keeping exact attention on the GPU. Offloading the KV cache to CPU memory to recover this batch headroom is thus a prerequisite for high-throughput block dLLM serving as well.

B. Problem: PCIe-Bound KV Fetching Bottleneck

Block dLLMs can adopt the sparse-attention-based KV offloading systems designed for AR LLMs [18], [31], [37]. As Section II-B describes, keeping exact attention under offloading is impractical because full-KV fetching is bound by PCIe bandwidth. We measure the pinned-host-to-GPU transfer time of the full-context KV cache and scale it across all layers and $T = 20$ per-step fetches for a $B = 32$ block. This gives a full-KV fetching ceiling of only 33 tokens/s, far below the full-KV GPU model forward throughput of 1.95k tokens/s. Fetching a

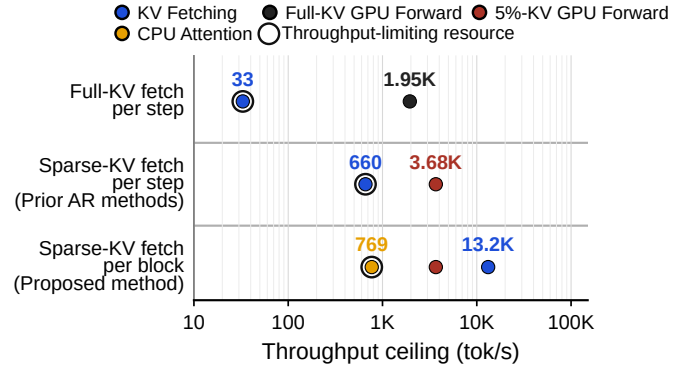


Fig. 5: Resource throughput ceilings for LLaDA 2.0-mini at a 32K context. Colored points denote KV-fetching, GPU-attention, and CPU-attention ceilings; an outer ring marks the throughput-limiting resource.

5% sparse KV cache reduces the transferred bytes by $20\times$ and raises the per-step ceiling to 660 tokens/s. However, this remains substantially below the 5%-KV GPU model forward throughput of 3.68k tokens/s.

Moreover, per-step critical-KV fetching still falls short of the ideal throughput of sparse attention under offloading, because KV fetching recurs at every step and its PCIe transfer cannot be fully hidden. This ideal, defined in Section II-B (Figure 3(d)), is the upper bound in which both selection and fetching hide behind the model forward pass, leaving latency bounded by the sparse-attention forward alone. Reaching it requires the critical-KV prefetch for layer $l+1$ to fit under the model forward time of layer l , which is difficult even with a small critical KV cache: as sparsity shrinks the transfer time, it also shrinks the window available for hiding, since sparse attention accelerates the forward pass itself and leaves only a short, FFN-dominated span per layer. Figure 5 quantifies this: at a 5% KV budget, the per-step fetching ceiling of 660 tokens/s falls far short of the measured GPU model forward throughput of 3.68k tokens/s, the upper bound that the ideal execution targets. The unhidden prefetch time is therefore exposed on the critical path at every step.

C. Proposed Solution: KV Selection Phase on CPU, Denoising Phase on GPU

Block dLLMs provide a structural opportunity to eliminate this bottleneck: their sparse-attention inference separates into a *KV selection phase* and a *denoising phase*. The KV selection phase computes exact attention over the full KV cache once per block and selects the critical KV cache that the block will attend to; the denoising phase denoises the block for the remaining steps using only the selected critical KV cache. Prior sparse-attention methods for fully bidirectional dLLMs, including SparseD and Sparse-dLLM, exploit cross-step attention consistency by deriving sparse KV index sets early and reusing them in later denoising steps [29], [33]. MAGE [16] first introduces this phase separation to block dLLMs: it performs full-KV selection at the first forward step

of a block and denoises with the selected set for the remaining $T-1$ steps. This style of sparse attention is also known to be more accurate than the per-step KV selection of AR serving systems [16], [32].

We propose to exploit this separation by mapping the selection phase to the CPU and the denoising phase to the GPU. This mapping resolves the KV fetching bottleneck of Section III-B, because the critical KV cache crosses PCIe only once per block. Moving from per-step to per-block fetching reduces the transferred data to $1/T$ and raises the KV fetching ceiling by $T\times$, to 13.2k tokens/s at a 5% budget with $T=20$ (Figure 5). This ceiling far exceeds the GPU model forward throughput (3.68k tokens/s), so KV fetching is no longer the bottleneck of system throughput.

The mapping also matches the resource characteristics of the two devices. In memory capacity, the selection phase requires the full KV cache and corresponds to the large CPU DRAM, while the denoising phase requires only the critical KV cache, 1–5% of the full KV cache, and fits the small GPU HBM. In compute capability, the difference in execution frequency compensates for the device gap. The CPU’s compute throughput is an order of magnitude lower than the GPU’s, but the selection phase runs only once per block while the denoising phase runs T times. When the two phases overlap, the CPU therefore has to complete just one selection within the time the GPU executes T denoising steps, so the time budget available to the CPU is T times the time the GPU spends on a single step.

D. Challenges of the Proposed Solution

Realizing the proposed mapping—executing the KV selection phase on the CPU and the denoising phase on the GPU—requires solving two challenges. Placing the MAGE algorithm on the two devices as-is prevents the two phases from overlapping and makes the CPU selection phase the new bottleneck.

1) **Challenge 1: Sequential Dependency between the Selection and Denoising Phases:** MAGE’s algorithm completes the selection phase within a block and then hands the selected critical KV cache to the denoising phase. Because of this sequential dependency, even if the selection phase runs on the CPU and the denoising phase on the GPU, the two computations are serialized and cannot overlap. In this case, if the CPU’s selection-phase time matches the GPU’s denoising-phase time, the final throughput is at most half of what full overlap allows. A new method is therefore needed to make the two computations overlappable.

2) **Challenge 2: Compute-Bound KV Selection on the CPU:** The KV selection phase on the CPU is compute-bound, becoming the new bottleneck of the decode throughput. KV selection computes exact attention over the full KV cache, and Figure 6 places this workload on the rooflines of three CPU platforms. The operational intensity of exact attention equals the number of query tokens that share one read of the KV cache. In AR LLMs, a single query shares each KV read, so the intensity ($G=4$ with GQA) stays below

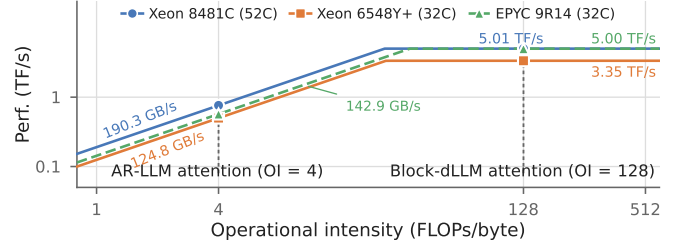


Fig. 6: Roofline analysis of AR-LLM and block-dLLM attention on three CPU platforms. The roofs are the measured DRAM bandwidth (GB/s) and the measured BF16+FMA compute peak (TF/s), the harmonic mean of the AVX-512 BF16-dot and FP32-FMA peaks that exact attention uses. Both evaluated models, LLaDA 2.0-mini & SDAR-8B ($G=4$), share the two workload positions.

the ridge points of all three platforms (26–35 FLOPs/byte), and the computation is memory-bound. In block dLLMs, the selection phase instead processes the $B=32$ [MASK] queries of a block in GEMM form, raising the intensity by $B\times$ to $BG=128$, beyond every ridge point, where the CPU’s compute throughput caps it. As a result, even with a time window of about T steps, the CPU attention ceiling is only 769 tokens/s (Figure 5), far below the GPU model forward throughput of the denoising phase (3.68k tokens/s). Preventing this requires reducing the selection phase’s computation to fit the CPU’s compute capability.

IV. HERALD

HERALD is a serving system that realizes the solution proposed in Section III-C: executing the KV selection phase on the CPU and the denoising phase on the GPU. HERALD realizes this mapping through an algorithm–system co-design for high-throughput block dLLM serving. Section IV-A presents the algorithm design, which adapts MAGE’s selection algorithm to resolve the two challenges of the mapping. Section IV-B then presents the system design that realizes the algorithm as an end-to-end serving system.

A. HERALD Algorithm Design

To resolve the two challenges in Section III-D, HERALD adapts MAGE’s algorithm to the CPU–GPU offloading environment. Draft-based lookahead (Section IV-A1) breaks the sequential dependency between the selection and denoising phases, resolving Challenge 1, and representative-query selection (Section IV-A2) reduces the computation of the selection phase to fit the CPU’s compute capability, resolving Challenge 2.

1) **Draft-Based Inter-Block Lookahead:** HERALD overlaps block $n+1$ ’s selection phase with block n ’s denoising phase by using a draft of block n , built from its step-0 logits, as a temporary prefix. This breaks the sequential dependency of Challenge 1.

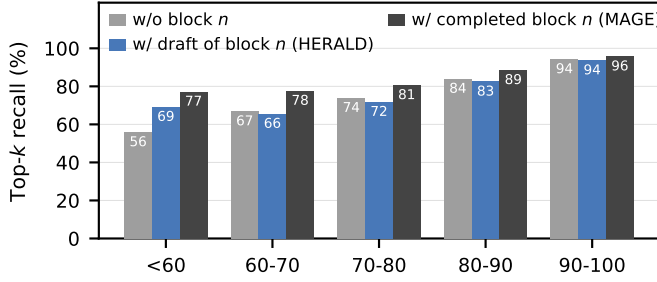


Fig. 7: Top- k recall of block $n+1$'s selection against the per-step oracle with block n omitted, drafted, or completed, bucketed by the recall with block n omitted ($k=1024$, LLaDA 2.0-mini, LongBench).

The query of the selection phase is always an all-`[MASK]` block in which every token is `[MASK]`. The selection query for block $n+1$ is therefore fixed by the `[MASK]` token embedding and block $n+1$'s token positions, independent of block n 's output.

This makes it possible to run block $n+1$'s selection phase in advance, independently of block n . The strategy selects the critical KV cache from the prefix before block n , and unconditionally includes block n in the critical KV cache once its generation completes. Block n 's denoising phase and block $n+1$'s selection phase then overlap.

In some cases, however, this approximation loses selection accuracy, because block $n+1$'s queries cannot reflect the completed block n . For example, when block n establishes a new topic of the document, block $n+1$ fails to retrieve the critical KV cache for that topic without block n . In our measurements, the recall of selection performed without block n drops below 0.6 at such boundaries, leaving an average gap of 21.3%p from the selection that uses the completed block n (Figure 7).

HERALD closes this gap by building a proxy for block n 's content from its step-0 logits. At step 0 the model produces logits for all B positions, but the confidence-based scheduler accepts only a subset of them to denoise. HERALD takes the argmax at every position, including the logits that would otherwise be discarded, to form a full-block draft of block n . The draft carries block n 's semantic information and guides block $n+1$'s selection: it recovers the recall of the collapsed boundaries by 13.3%p on average, closing more than half of the gap to the completed block (up to +28.5%p), while remaining neutral elsewhere. Section IV-A3 shows that a small increase of the sparsity budget recovers the remaining average loss.

Through this modification, the CPU's selection phase runs overlapped with the GPU's denoising phase. The draft requires no separate model forward pass, and the CPU waits only for the first two steps of the GPU's denoising phase. The argmax of the logits produced by the denoising phase's first step forms the draft block tokens, and the KV cache of the draft is computed by batching it with the denoising phase's

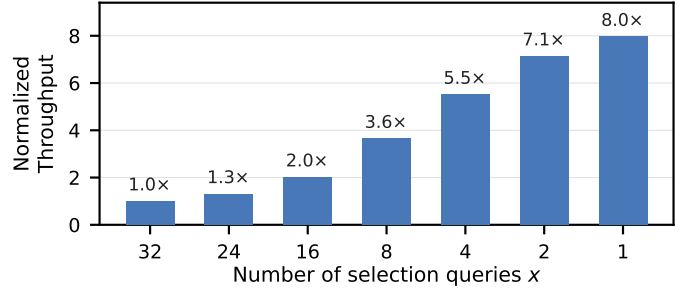


Fig. 8: Measured throughput of the selection kernel as the number of queries is reduced from $B=32$ to one, normalized to $x=32$ (CPU, 32K context, 5% budget).

second step. Once the draft block's KV cache is delivered to the CPU, the selection phase begins and overlaps with the remaining steps of the denoising phase.

2) **Representative-Query Selection:** HERALD reduces the selection phase's B -query attention to a single representative query: the `[MASK]` token at the center of the block. This resolves the CPU compute bottleneck of Challenge 2.

The selection phase is slow on the CPU because of its B queries. In the selection phase, B queries perform attention in GEMM form, and the query count, enlarged relative to AR LLMs, makes this computation compute-bound on the CPU. Accelerating it therefore requires reducing the number of queries.

The structure of the selection phase makes this reduction possible. At the moment selection is performed, all B positions of the next block carry the identical `[MASK]` token embedding and differ only in their positions within the block. The attention scores of the B queries therefore differ only through their positional encodings: under RoPE, the score between block position i and a prefix key $\mathbf{k}_s$ at position s is

$$\text{score}(i, s) = (R_{\theta, p_i} \mathbf{q})^\top (R_{\theta, s} \mathbf{k}_s) = \mathbf{q}^\top R_{\theta, p_i - s} \mathbf{k}_s, \quad (1)$$

where the pre-RoPE query $\mathbf{q} = W_Q \mathbf{e}_{[\text{MASK}]}$ is shared by all B positions and $R_{\theta, \delta}$ is the RoPE rotation for relative offset $\delta = p_i - s$. The scores across the B queries thus differ only through this relative offset, and even this difference is confined within the block, at most $B/2=16$ from the block center, bounded regardless of context length. The prefix rankings of the B queries are correspondingly redundant, so the query count can be reduced with only a small loss in selection accuracy.

HERALD uses the `[MASK]` token at the center of the block as the representative. Since the queries differ only in their relative offsets to the prefix tokens, the representative should be the position whose offsets deviate least from those of the entire block, and the center position p_c minimizes this deviation:

$$p_c = \arg \min_{p_i} \sum_{s \in \text{prefix}} (p_i - s - \bar{\delta})^2, \quad (2)$$

where $\bar{\delta}$ denotes the mean relative offset between the block positions and the prefix tokens. Switching to the representative

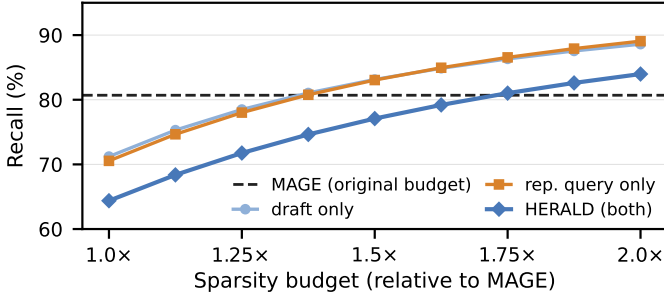


Fig. 9: Selection recall against the per-step oracle as the sparsity budget grows, for each modification alone and for HERALD combining both, with MAGE at the original budget as the reference ($k=1024$, LLaDA 2.0-mini, LongBench).

query preserves the nature of the selection: at every layer, the CPU still scores every entry in the full prefix KV cache, so only the query used for ranking changes, and no prefix KV entry is summarized or omitted.

The throughput gain of query reduction continues monotonically down to $x=1$. The cost of the selection kernel includes not only the attention computation but also the storing of attention scores and the top- k selection, all of which shrink in proportion to the query count. Figure 8 shows the measured throughput of the selection kernel as a function of the query count: reducing the queries from 32 to one raises the kernel throughput by $8\times$. HERALD therefore reduces the queries all the way to $x=1$. Section IV-B4 describes how HERALD realizes this selection as a fused attention and top- k kernel on the CPU.

The recall loss of this reduction is likewise recovered by the sparsity budget in Section IV-A3.

3) Recovering Selection Quality at Low System Cost:

HERALD recovers MAGE’s selection quality by raising the sparsity budget. Figure 9 shows the selection recall of each variant as the budget grows: each modification alone surpasses MAGE’s original-budget recall at about a $1.4\times$ budget, and HERALD, combining both, recovers it within $1.75\times$. This factor is relative to MAGE’s operating budget: at whatever budget MAGE runs, HERALD matches its selection quality by provisioning about $1.75\times$ that budget.

This budget increase costs only batch size. The per-request critical KV cache grows by $1.75\times$, shrinking the maximum batch size by the same factor. This batch reduction causes only a small throughput loss: after offloading, the batch already sits near the GPU’s compute-saturation point, so the denoising phase keeps operating in the saturated regime and loses only about 17% of its throughput in our measurement.

The throughput gains of the two modifications are far larger: draft-based lookahead overlaps selection with the remaining denoising steps, and the representative query raises the selection kernel’s throughput by $8\times$. The next section describes the end-to-end execution that combines the two techniques.

B. HERALD System Design

1) **System Overview:** HERALD implements the algorithm of Section IV-A as a dual-stream execution. The main stream executes block n ’s denoising phase, and the retrieval stream executes block $n+1$ ’s selection phase. The main stream runs entirely on the GPU; in the retrieval stream, the CPU performs the full-KV selection and the GPU executes the FFN, forming a per-layer pipeline.

The KV cache is organized into two levels: GPU HBM holds double-buffered sparse KV pools, and CPU DRAM holds the full KV pool. The sparse KV pools consist of an active pool and an idle pool, and the two pool pointers are swapped at every block transition. The main stream denoises block n using the KV cache in the active sparse pool, while the retrieval stream selects block $n+1$ ’s critical KV cache from the full KV pool and moves it into the idle sparse pool.

The two streams run overlapped and synchronize at each block transition, so the end-to-end throughput is bound by the slower stream. The main stream performs T steps of model forwarding per block, whereas the retrieval stream performs one forward pass per block with a single query. HERALD maximizes throughput by fitting the retrieval stream within the main stream’s time window.

This section describes the system top-down. Section IV-B2 details the end-to-end execution flow formed by the two streams. Section IV-B3 zooms into the retrieval stream and describes its layer-wise CPU–GPU cooperative pipeline. Section IV-B4 describes the implementation of the KV selection kernel that the CPU executes in this pipeline.

2) **End-to-End Execution Flow:** The prefill phase generates the KV cache of the input prompt and stores it in the CPU full KV pool and the GPU sparse KV pool. For each layer, HERALD copies the entire generated KV cache asynchronously to CPU DRAM (Figure 10, left) and performs KV cache selection with the B [MASK] tokens forwarded together with the prompt, selecting the layer’s critical KV cache. The selected entries are loaded into the idle sparse pool in GPU memory, and the rest are freed from the GPU. The first two layers always perform exact attention during denoising to preserve accuracy, so their full KV cache is kept separately in GPU memory.

In the decode phase, the main stream is responsible for block n ’s denoising phase. The main stream executes step 0 with confidence-based token acceptance (1) and forms block n ’s full-block draft from the argmax predictions of the remaining masked positions (2). The next denoising step runs as a $2B$ -token forward pass fused with the draft’s KV computation (3, Section IV-A1). Within this forward pass, the denoising group and the draft group use different custom attention masks: both groups attend to the KV cache of the completed prefix but not to each other. The draft KV cache produced by this forward pass is transferred to CPU memory, where the retrieval stream consumes it. The main stream then continues denoising until every position in block n is an accepted token position (4). Once the block is completed, a forward pass over the completed block computes its KV cache for prefix

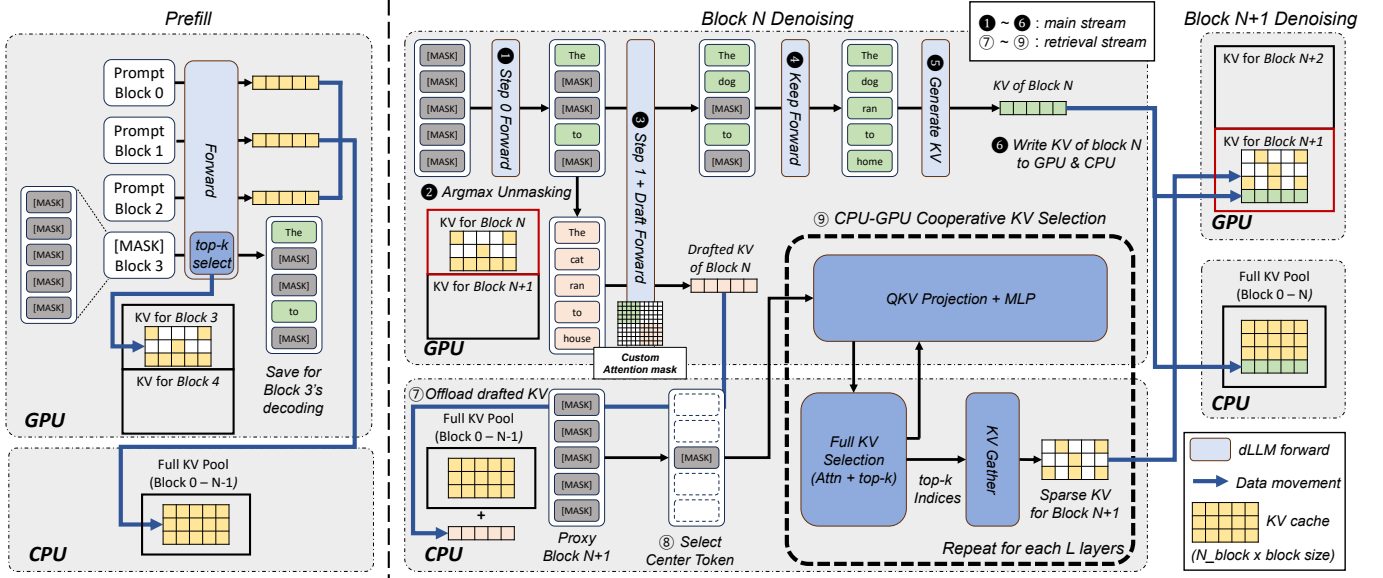


Fig. 10: System overview of HERALD. **Left:** the prefill phase appends a [MASK] block to the prompt, offloads the KV cache to CPU DRAM layer-wise, and performs top- k selection to initialize the sparse KV pool. **Right:** the decode phase operates two concurrent streams with double-buffered KV pools; the retrieval stream writes the next block’s entries to the idle pool while the main stream denoises from the active pool.

caching (5); it is written to both block $n+1$ ’s GPU pool and the full KV pool in CPU DRAM, replacing the draft (6).

The retrieval stream starts block $n+1$ ’s selection phase as soon as the drafted KV arrives. It temporarily appends the draft KV cache to the full KV pool (7) and uses the center [MASK] position of block $n+1$ as the representative query (8). With this query, the retrieval stream performs selection layer by layer, writing the top- k entries into the idle GPU sparse pool (9), and this execution overlaps with the remaining denoising steps of the main stream.

The draft affects only the selection and never the generated output. Once block n is completed, its accurate KV cache replaces the draft, so block $n+1$ ’s denoising and block $n+2$ ’s selection use the accurate KV cache of the completed block. Draft errors therefore never propagate into the KV cache used for subsequent steps.

3) CPU–GPU Cooperative Pipeline for the Retrieval Stream: The retrieval stream divides each layer’s selection-phase computation between the CPU and the GPU. For each Transformer layer, the GPU computes the QKV projection of the representative query, the CPU performs the KV selection over the CPU-resident KV cache, and the attention output returns to the GPU for the FFN (Figure 11). The KV selection consists of full-KV attention and top- k selection. Unlike the main stream’s B -query denoising, the query is a single token, so the cost of the GPU-side projection and FFN is small. These operations execute serially within a layer because each consumes the preceding operation’s output.

Once the CPU’s full-KV selection produces a layer’s top- k entries, the CPU gathers the selected entries into a contiguous pinned buffer while the GPU executes that layer’s FFN. The

selected entries are scattered across CPU memory, so the gather performs random DRAM reads whose volume scales with the batch size; overlapping it with the FFN hides most of this cost.

Once the gather completes, the CPU issues an asynchronous DMA transfer to the idle GPU sparse pool, and this transfer overlaps with the next layer’s operations. The per-layer transfer time of the critical KV cache is therefore hidden under the retrieval stream’s CPU KV selection and FFN computation, and the PCIe bottleneck of KV transfer described in Section III-B is eliminated.

4) Fused CPU Selection Kernel: The CPU computation of the selection phase runs as a kernel that fuses attention and top- k selection into a single streaming pass over the KV cache. For the kernel, S is the number of prefix KV entries and k is the per-KV-head selection budget. The index h identifies a KV head, and G is the number of query heads that share one KV head under GQA. The indices $g \in \{1, \dots, G\}$ and $s \in \{1, \dots, S\}$ identify a query head within the GQA group and a prefix position, respectively.

The challenge is that while attention can be computed tile-by-tile without storing the full score matrix, top- k selection must compare scores across all S prefix positions. Storing the full score matrix in DRAM during attention and re-reading it at top- k time is expensive; under GQA, running G separate top- k passes, one per query head, multiplies this cost.

HERALD addresses this with two key design choices. First, the kernel manages scores at the granularity of a single KV head: for each KV head h , it streams the KV cache from CPU memory in tiles, accumulating the raw attention scores $s[g, s]$ for all G query heads alongside the FlashAttention [5]

TABLE I: LongBench accuracy across both models and four tasks at KV cache budgets from 1% to 20%. The Avg. column shows the difference from Dense in parentheses. Bold rows mark the Dense reference and the operating budget used in the throughput evaluation for each method (MAGE-Offload 2.5%, HERALD 5%, InfiniGen 20%).

Method	LLaDA 2.0-mini 16B					SDAR-8B-Chat				
	2WikiMQA	NarrativeQA	QMSum	RepoBench-P	Avg.	2WikiMQA	NarrativeQA	QMSum	RepoBench-P	Avg.
Dense	48.88	26.69	22.59	56.58	38.69	44.37	28.48	23.98	48.57	36.35
MAGE-Offload (1%)	49.62	26.48	23.26	55.96	38.83 (+0.14)	43.07	28.76	23.15	44.72	34.92 (−1.43)
MAGE-Offload (2.5%)	49.52	26.89	22.92	56.15	38.87 (+0.19)	43.40	29.19	23.38	47.42	35.85 (−0.50)
MAGE-Offload (5%)	49.03	27.06	22.74	56.91	38.94 (+0.25)	43.32	28.68	23.55	47.81	35.84 (−0.51)
MAGE-Offload (10%)	49.96	27.24	22.94	56.66	39.20 (+0.52)	44.53	29.81	24.01	48.04	36.60 (+0.25)
InfiniGen (2.5%)	21.50	14.86	19.70	33.11	22.29 (−16.39)	10.79	6.42	12.82	24.02	13.51 (−22.84)
InfiniGen (5%)	28.68	19.51	21.29	39.53	27.25 (−11.43)	22.86	16.66	16.28	26.84	20.66 (−15.69)
InfiniGen (10%)	37.00	19.21	21.21	45.38	30.70 (−7.98)	24.27	18.29	19.55	28.94	22.76 (−13.59)
InfiniGen (20%)	45.10	23.09	22.59	51.28	35.52 (−3.17)	34.17	22.14	22.22	29.55	27.02 (−9.33)
HERALD (1%)	48.55	25.91	21.98	54.74	37.80 (−0.89)	40.34	28.26	22.09	41.83	33.13 (−3.22)
HERALD (2.5%)	49.87	25.61	21.56	55.45	38.12 (−0.56)	43.06	28.87	21.66	44.68	34.57 (−1.78)
HERALD (5%)	50.06	27.30	21.94	55.78	38.77 (+0.09)	43.24	29.75	22.09	46.70	35.45 (−0.91)
HERALD (10%)	49.67	26.70	21.51	55.78	38.41 (−0.27)	43.72	29.04	22.60	47.27	35.66 (−0.69)

TABLE II: Peak decode throughput (tokens/s) with the batch size that maximizes it.

Ctx	Method	LLaDA 2.0-mini 16B		SDAR-8B-Chat	
		bs	Throughput	bs	Throughput
16K	Dense	32	1864 (1.00×)	15	754 (1.00×)
	InfiniGen	16	555 (0.30×)	16	178 (0.24×)
	MAGE-Offload	96	749 (0.40×)	64	230 (0.31×)
	HERALD	160	3385 (1.82×)	68	1367 (1.81×)
32K	Dense	16	1168 (1.00×)	8	458 (1.00×)
	InfiniGen	16	340 (0.29×)	16	89 (0.19×)
	MAGE-Offload	56	403 (0.34×)	40	125 (0.27×)
	HERALD	80	2665 (2.28×)	34	867 (1.89×)

B. Accuracy Evaluation

HERALD reaches near-lossless accuracy at a 5% KV budget, with the average difference from Dense within one point on both models (LLaDA +0.1, SDAR −0.9). We evaluate on four LongBench [2] tasks (2WikiMQA, NarrativeQA, QMSum, RepoBench-P), covering multi-hop QA, single-document QA, summarization, and code completion; Table I shows the results across KV cache budgets from 1% to 20%. MAGE-Offload recovers Dense accuracy at 2.5%, while InfiniGen stays −3.2/−9.3 points below Dense even at 20%. All three methods select once per block and reuse the selection, so the gap comes from selection accuracy: errors of proxy-based selection (InfiniGen) persist through every denoising step of the block due to the reuse, whereas full-KV selection (MAGE, HERALD) chooses a reliable block-wide critical KV cache. HERALD adds the representative-query and draft-prefix approximations, and the selection quality they lose is recovered with a few additional percentage points of budget relative to MAGE.

C. Throughput Evaluation

We use peak decode throughput as the primary performance metric for throughput results. We use the smallest KV budget that reaches near-lossless accuracy for MAGE (2.5%) and

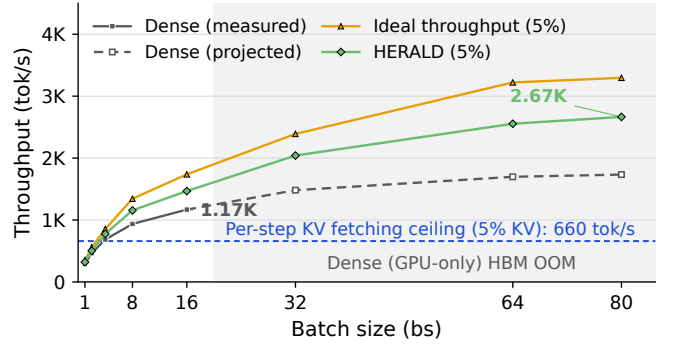


Fig. 12: Maximum measured throughput on LLaDA 2.0-mini at 32K with $T=20$. Dense reaches 1.17K tok/s at $bs=16$ before HBM OOM, whereas HERALD with 5% KV reaches 2.67K tok/s at $bs=80$ (2.28 $\times$). The blue dashed line shows the analytical PCIe ceiling for per-step 5%-KV transfer.

HERALD (5%), as established in Section VI-B. InfiniGen does not reach near-lossless accuracy within the evaluated range, so we use its largest evaluated budget (20%), favoring its accuracy at the cost of greater transfer and sparse-attention work.

HERALD achieves the highest decode throughput on both models and both context lengths, and its gain over Dense grows with context: 1.81–1.82 $\times$ at 16K and 1.89–2.28 $\times$ at 32K. Table II reports the throughput-maximizing batch size and the resulting peak decode throughput. Dense is limited to batch sizes of 15–32 at 16K and 8–16 at 32K because the KV cache occupies HBM. InfiniGen and MAGE-Offload accommodate larger batches through offloading, but their selection and retrieval overheads are exposed on the critical path by the PCIe bottleneck, leaving them below Dense at 0.19–0.30 $\times$ and 0.27–0.40 $\times$. HERALD scales the batch size up to 160 while retaining Dense-level per-block efficiency, reaching up to 2.28 $\times$ Dense’s throughput.

Figure 12 shows the structure of this gain with a batch

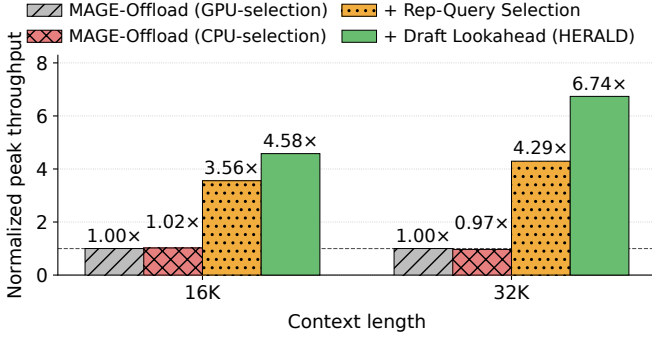


Fig. 13: Peak-throughput ablation of HERALD’s design components on LLaDA 2.0-mini 16B at 16K and 32K context lengths ($T = 20$, 5% KV selection). Each bar reports the peak decode throughput over the feasible batch sizes, normalized to MAGE-Offload with GPU selection at the same context length.

sweep on LLaDA 2.0-mini at 32K. Dense stops at 1.17K tokens/s at $bs=16$ due to HBM OOM. Even with the small 5%-budget critical KV cache, per-step fetch is capped by the 660 tokens/s PCIe ceiling, below Dense (Section III-B). HERALD fetches the critical KV cache once per block, raising this ceiling by $T\times$ and breaking through it: it scales the batch to 80 and reaches 2.67K tokens/s. This exceeds the projected Dense line without the HBM capacity limit ($\sim 1.7K$) and approaches the ideal throughput, the upper bound in which only the 5%-budget sparse attention runs on the GPU (Section II-B).

D. Ablation Study

Figure 13 quantifies how each HERALD component improves peak throughput, with every configuration evaluated at its best measured feasible batch size. Moving the 32-query full-KV selection from the GPU to the CPU eliminates the full-KV PCIe transfer, but achieves only 755 and 382 tokens/s, approximately matching the GPU-selection baseline of 739 and 396 tokens/s. Thus, CPU mapping alone merely shifts the bottleneck from PCIe transfer to expensive 32-query CPU selection. Representative-query selection provides the first major gain, raising throughput to 2,630 and 1,699 tokens/s— $3.48\times$ and $4.45\times$ over the preceding CPU-selection configuration. Draft-based lookahead then contributes an additional $1.29\times$ and $1.57\times$ by overlapping selection with the current block’s denoising. Together, the two techniques achieve 3,385 and 2,665 tokens/s, corresponding to $4.58\times$ and $6.74\times$ the peak throughput of MAGE-Offload at 16K and 32K, respectively.

Figure 14 evaluates HERALD across different numbers of denoising steps. At $T=8$, an aggressive decoding point that produces four tokens per step ($B/T=4$), HERALD matches the already-fast Dense baseline; at every larger tested T , it achieves higher throughput. The latency breakdown shows that the bottleneck shifts from retrieval to the main stream as T increases: once-per-block retrieval latency remains nearly constant, whereas main-stream latency grows with T . The

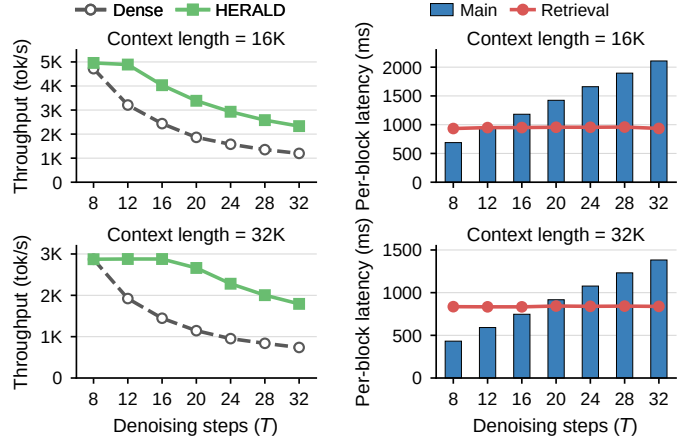


Fig. 14: Sensitivity to denoising steps T on LLaDA 2.0-mini with 5% KV. Rows show 16K and 32K; columns show throughput and HERALD’s stream latency. Dense/HERALD use $bs=32/160$ at 16K and $16/80$ at 32K. Retrieval latency remains nearly constant, while Main latency grows with T , explaining the resulting throughput trend.

retrieval-bound regime leaves further headroom for faster CPU selection kernels or using faster CPUs.

VII. RELATED WORK

Acceleration Methods of Diffusion LLMs. Prior dLLM systems accelerate inference through parallel or adaptive sampling [34], [36] or reuse intermediate features and KV states [21], [23]. For fully bidirectional dLLMs, SparseD and Sparse-dLLM exploit cross-step attention consistency; Sparse-dLLM further combines sparse attention with delayed bidirectional cache eviction [29], [33]. MAGE performs full-KV selection at the initial all-[MASK] step of each block and reuses the selected indices across subsequent denoising steps [16]. These methods reduce denoising work or the GPU-resident KV footprint, whereas HERALD targets a different bottleneck: selecting and retrieving a critical subset from the full prefix KV cache stored in CPU memory.

KV Offloading in LLM Serving. KV offloading systems divide by the inference phase they target. Prefill-side systems build a prefix cache: the KV cache of completed or shared prefixes is stored in CPU DRAM or storage and restored to the GPU when the prefix recurs, avoiding prefill recomputation [6], [20], [26]. Decoding still runs with the restored KV cache resident in HBM, so the capacity limit of Section III-A remains. HERALD instead targets the decode phase, offloading the KV cache of active requests to scale the batch size, following the AR decode-time offloading line of Section II-B [18], [31], and redesigns it around the two-phase KV access pattern of block dLLMs.

VIII. CONCLUSION

Block dLLMs require KV offloading at long contexts, but AR-style offloading systems select and fetch the critical KV cache at every step and remain capped by per-step KV

fetching over PCIe. This paper showed that block dLLM sparse attention separates inference into a KV selection phase and a denoising phase, and that mapping the two phases onto the CPU and the GPU removes KV fetching as the bottleneck. HERALD realizes this mapping with draft-based lookahead, representative-query selection, and a dual-stream pipeline over double-buffered sparse KV pools. On two production block dLLMs, HERALD reaches near-lossless accuracy at a 5% KV budget and up to $2.28\times$ the decode throughput of a Dense GPU baseline.

REFERENCES

- [1] M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov, "Block diffusion: Interpolating between autoregressive and diffusion language models," in *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. [Online]. Available: <https://openreview.net/forum?id=tyEyYT267x>
- [2] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3119–3137. [Online]. Available: <https://aclanthology.org/2024.acl-long.172/>
- [3] T. Bie, M. Cao, K. Chen, L. Du, M. Gong, Z. Gong, Y. Gu, J. Hu, Z. Huang, Z. Lan, C. Li, C. Li, J. Li, Z. Li, H. Liu, L. Liu, G. Lu, X. Lu, Y. Ma, J. Tan, L. Wei, J.-R. Wen, Y. Xing, X. Zhang, J. Zhao, D. Zheng, J. Zhou, J. Zhou, Z. Zhou, L. Zhu, and Y. Zhuang, "Llada2.0: Scaling up diffusion language models to 100b," 2025. [Online]. Available: <https://arxiv.org/abs/2512.15745>
- [4] S. Cheng, Y. Bian, D. Liu, L. Zhang, Q. Yao, Z. Tian, W. Wang, Q. Guo, K. Chen, B. Qi, and B. Zhou, "Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation," 2025. [Online]. Available: <https://arxiv.org/abs/2510.06303>
- [5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: fast and memory-efficient exact attention with io-awareness," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [6] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "Cost-efficient large language model serving for multi-turn conversations with cachedattention," in *Proceedings of the 2024 USENIX Conference on Usenix Annual Technical Conference*, ser. USENIX ATC'24. USA: USENIX Association, 2024.
- [7] S. Gong, S. Agarwal, Y. Zhang, J. Ye, L. Zheng, M. Li, C. An, P. Zhao, W. Bi, J. Han, H. Peng, and L. Kong, "Scaling diffusion language models via adaptation from autoregressive models," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=j1tSLYKwg8>
- [8] S. Gong, M. Li, J. Feng, Z. Wu, and L. Kong, "Diffuseq: Sequence to sequence text generation with diffusion models," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=jQj-rLVXsj>
- [9] Google DeepMind, "Gemini diffusion," <https://deepmind.google/models/gemini-diffusion/>, 2025, accessed: 2026-04-14.
- [10] JEDEC, "DDR5 SDRAM Standard (JESD79-5)," <https://www.jedec.org/standards-documents/docs/jesd79-5>, 2020.
- [11] X. Jiang, Y. Zhou, S. Cao, I. Stoica, and M. Yu, "Neo: Saving gpu memory crisis with cpu offloading for online llm inference," in *Proceedings of Machine Learning and Systems*, M. Zaharia, G. Joshi, and Y. Lin, Eds., vol. 7. MLSys, 2025. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2025/file/66a026c0d17040889b50f0dfa650e5e0-Paper-Conference.pdf
- [12] H. Kim, N. Wang, Q. Xia, J. Huang, A. Yazdanbakhsh, and N. S. Kim, "Lia: A single-gpu llm inference acceleration with cooperative amx-enabled cpu-gpu computation and cxl offloading," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 544–558. [Online]. Available: <https://doi.org/10.1145/3695053.3731092>
- [13] J. Kim, K. Shah, V. Kontonis, S. Kakade, and S. Chen, "Train for the worst, plan for the best: Understanding token ordering in masked diffusions," 2025. [Online]. Available: <https://arxiv.org/abs/2502.06768>
- [14] K. Kim, O. Kwon, Y. Park, and J. W. Lee, "AiDE: Attention-FFN Disaggregated Execution for Cost-Effective LLM Decoding on CXL-PNM," *IEEE Computer Architecture Letters*, vol. 24, no. 02, pp. 285–288, Jul. 2025. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/LCA.2025.3597323>
- [15] M. Kim, C. Xu, C. Hooper, H. Singh, B. Athiwaratkun, C. Zhang, K. Keutzer, and A. Gholami, "CdLm: Consistency diffusion language models for faster sampling," 2026. [Online]. Available: <https://arxiv.org/abs/2511.19269>
- [16] O. Kwon, Y. Kim, D. Kim, M. Kim, Y. Park, and J. W. Lee, "Mage: All-[mask] block already knows where to look in diffusion llm," 2026. [Online]. Available: <https://arxiv.org/abs/2602.14209>
- [17] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [18] W. Lee, J. Lee, J. Seo, and J. Sim, "Infinigen: efficient generative inference of large language models with dynamic kv cache management," in *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'24. USA: USENIX Association, 2024.
- [19] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," 2023. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [20] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, and J. Jiang, "Lmcache: An efficient kv cache layer for enterprise-scale llm inference," 2025. [Online]. Available: <https://arxiv.org/abs/2510.09665>
- [21] Z. Liu, Y. Yang, Y. Zhang, J. Chen, C. Zou, Q. Wei, S. Wang, and L. Zhang, "dLlm-cache: Accelerating diffusion large language models with adaptive caching," 2025. [Online]. Available: <https://arxiv.org/abs/2506.06295>
- [22] A. Lou, C. Meng, and S. Ermon, "Discrete diffusion modeling by estimating the ratios of the data distribution," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [23] X. Ma, R. Yu, G. Fang, and X. Wang, "dKV-cache: The cache for diffusion language models," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=Gppo2JlmHs>
- [24] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. ZHOU, Y. Lin, J.-R. Wen, and C. Li, "Large language diffusion models," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=Knqic0znVF>
- [25] NVIDIA, "NVIDIA H100 Tensor Core GPU Datasheet," <https://resources.nvidia.com/en-us-gpu-resources/h100-datasheet-24306>, 2023, accessed: 2026-08-01.
- [26] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot," in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. Santa Clara, CA: USENIX Association, Feb. 2025, pp. 155–170. [Online]. Available: <https://www.usenix.org/conference/fast25/presentation/qin>
- [27] S. S. Sahoo, M. Arriola, A. Gokaslan, E. M. Marroquin, A. M. Rush, Y. Schiff, J. T. Chiu, and V. Kuleshov, "Simple and effective masked diffusion language models," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=L4uaAR4ArM>
- [28] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: high-throughput generative inference of large language models with a single gpu," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.
- [29] Y. Song, X. Liu, R. Li, Z. Liu, Z. Huang, Q. Guo, Z. He, and X. Qiu, "Sparse-dllm: Accelerating diffusion llms with dynamic cache eviction," 2025. [Online]. Available: <https://arxiv.org/abs/2508.02558>

- [30] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, “Dynamollm: Designing llm inference clusters for performance and energy efficiency,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, Mar. 2025, p. 1348–1362. [Online]. Available: <http://dx.doi.org/10.1109/HPCA61900.2025.00102>
- [31] H. Sun, L.-W. Chang, W. Bao, S. Zheng, N. Zheng, X. Liu, H. Dong, Y. Chi, and B. Chen, “Shadowkv: Kv cache in shadows for high-throughput long-context llm inference,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. ICML’25. JMLR.org, 2025.
- [32] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, “QUEST: Query-aware sparsity for efficient long-context LLM inference,” in *Forty-first International Conference on Machine Learning*, 2024. [Online]. Available: <https://openreview.net/forum?id=KzACYw0MTV>
- [33] Z. Wang, G. Fang, X. Ma, X. Yang, and X. Wang, “Sparsed: Sparse attention for diffusion language models,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=dwbrZtYP04>
- [34] Q. Wei, Y. Zhang, Z. Liu, P. Zeng, Y. Wang, B. Qi, D. Liu, and L. Zhang, “Accelerating diffusion large language models with slowfast sampling: The three golden principles,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=Uh17FiwF4q>
- [35] C. Wu, H. Zhang, S. Xue, S. Diao, Y. Fu, Z. Liu, P. Molchanov, P. Luo, S. Han, and E. Xie, “Fast-dLLM v2: Efficient block-diffusion LLM,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=1NZ3DHF9nT>
- [36] C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie, “Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.22618>
- [37] J. Xu, J. Pan, H. Wang, Y. Zhou, J. Ye, Y. Wang, and G. Dai, “Specontext: Enabling efficient long-context reasoning with speculative context sparsity in llms,” in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1832–1847. [Online]. Available: <https://doi.org/10.1145/3779212.3790224>
- [38] J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong, “Dream 7b: Diffusion large language models,” *arXiv preprint arXiv:2508.15487*, 2025. [Online]. Available: <https://arXiv.org/abs/2508.15487>
- [39] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, “Flashinfer: Efficient and customizable attention engine for LLM inference serving,” in *Eighth Conference on Machine Learning and Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=RXPOfAsL8F>
- [40] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “Sglang: efficient execution of structured language model programs,” in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS ’24. Red Hook, NY, USA: Curran Associates Inc., 2024.